

Projet Apprentissage Statistique - Gabriel Stremsdoerfer et Denis Jaouen

Table of contents

1	Traitement des données	2
1.1	Importation des données et librairies	2
1.2	Discrétisation de Y	2
2	Statistiques descriptives	11
3	Création d'un jeu de donnée imputé avec le package missmda	19
4	Stacking de XGboost et RF	19
4.1	Préparation des données	19
4.2	Séparation apprentissage / test	20
5	Paramètres de RF	20
5.1	Paramètres de XGBoost	22
6	Stacking	27
6.1	Préparation des folds	27
6.2	Méta-modèle multinomial	29
6.3	RF et XGBoost finaux	29
6.4	Prédictions du stacking	31
7	Premiers résultats	31
7.1	Classes prédites	31
7.2	Matrices de confusion	32
7.3	Comparaison des accuracies	32
7.4	Matrices de confusion pondérées utilisées plus tard	33
8	Analyse Post-hoc des modèles	33
8.1	3. OOB error du Random Forest	34
9	4. Partial Dependence Plot (PDP)	34
10	5. Accumulated Local Effects (ALE)	36
11	6. Permutation Feature Importance (PFI)	37
12	7. LOFO - Leave One Feature Out	40
13	8. Valeurs de Shapley : explication locale	42
13.1	Résultats finaux à comparer	43
14	Idées pour améliorer prédiction:	46

1 Traitement des données

1.1 Importation des données et librairies

```
setwd(getwd())
escap = read.csv("ESCAP.csv", sep = ';')
summary(escap)
```

A01		Q03		Q04		Q04A	
Min.	: 1	Min.	:1.000	Min.	:1.000	Min.	:1.000
1st Qu.	:10795	1st Qu.	:1.000	1st Qu.	:1.000	1st Qu.	:1.000
Median	:22545	Median	:1.000	Median	:1.000	Median	:1.000
Mean	:22534	Mean	:1.494	Mean	:1.042	Mean	:1.099
3rd Qu.	:34228	3rd Qu.	:2.000	3rd Qu.	:1.000	3rd Qu.	:1.000
Max.	:45562	Max.	:2.000	Max.	:2.000	Max.	:3.000
		NA's	:23	NA's	:1583	NA's	:593

Q04B		Q05		Q06A		Q06B	
Min.	:1.000	Min.	:1.000	Min.	:1.000	Min.	:1.000
1st Qu.	:1.000	1st Qu.	:1.000	1st Qu.	:1.000	1st Qu.	:1.000
Median	:2.000	Median	:1.000	Median	:1.000	Median	:1.000
Mean	:2.177	Mean	:1.302	Mean	:1.154	Mean	:1.288
3rd Qu.	:2.000	3rd Qu.	:2.000	3rd Qu.	:1.000	3rd Qu.	:2.000
Max.	:4.000	Max.	:2.000	Max.	:3.000	Max.	:3.000
NA's	:12777	NA's	:40	NA's	:34	NA's	:40

Q08		Q08C		Q09A1		Q09B1		Q10A1	
Min.	:1.000	Min.	:1.00	Min.	:1.000	Min.	:1.000	Min.	:1.000
1st Qu.	:1.000	1st Qu.	:1.00	1st Qu.	:1.000	1st Qu.	:1.000	1st Qu.	:4.000
Median	:1.000	Median	:1.00	Median	:1.000	Median	:1.000	Median	:6.000
Mean	:1.145	Mean	:1.47	Mean	:1.601	Mean	:1.494	Mean	:5.219
3rd Qu.	:1.000	3rd Qu.	:2.00	3rd Qu.	:1.000	3rd Qu.	:1.000	3rd Qu.	:7.000
Max.	:5.000	Max.	:5.00	Max.	:7.000	Max.	:7.000	Max.	:9.000
NA's	:95	NA's	:81	NA's	:402	NA's	:256	NA's	:1536

Q10B1		Q19A		B08A		B08B	
Min.	:1.000	Min.	: 0.0	Min.	:1.000	Min.	:1.000
1st Qu.	:5.000	1st Qu.	:14.0	1st Qu.	:2.000	1st Qu.	:1.000
Median	:6.000	Median	:15.0	Median	:3.000	Median	:2.000
Mean	:5.636	Mean	:14.3	Mean	:2.944	Mean	:2.474
3rd Qu.	:6.000	3rd Qu.	:16.0	3rd Qu.	:4.000	3rd Qu.	:4.000
Max.	:9.000	Max.	:32.0	Max.	:5.000	Max.	:5.000
NA's	:1475	NA's	:2393	NA's	:1400	NA's	:1050

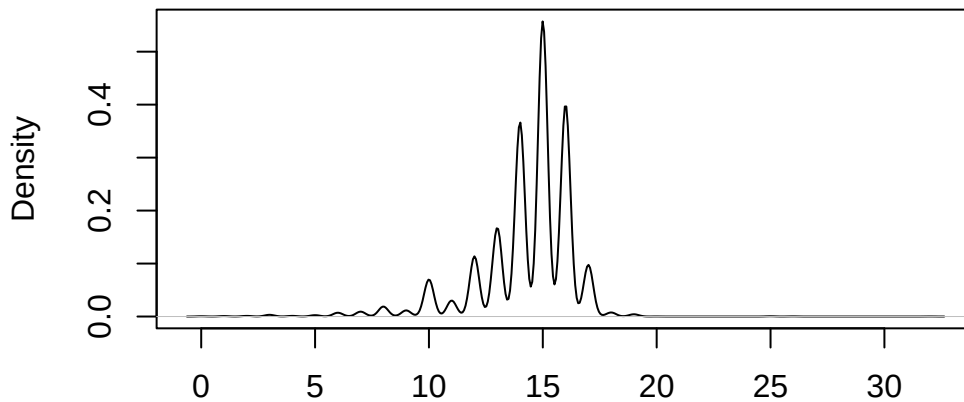
pm17B
Length:13314
Class :character
Mode :character

1.2 Discretisation de Y

```
escap$Y <- escap$Q19A
escap = escap[,-15]

plot(density(escap$Y, na.rm = TRUE))
```

density(x = escap\$Y, na.rm = TRUE)



N = 10921 Bandwidth = 0.2092

```
q <- quantile(
  escap$Y,
  probs = c(0.20, 0.40, 0.60, 0.80),
  na.rm = TRUE
)

escap$Y <- cut(
  escap$Y,
  breaks = c(-Inf, q, Inf),
  labels = c("13-", "14", "15", "16", "17+"),
  include.lowest = TRUE
)

escap$Y <- as.factor(escap$Y)

table(escap$Y)
```

```
13-  14  15  16  17+
2551 2150 3244 2331  645
```

La distribution de Y a des queues de distribution très aplaties, ce qui peut poser problème en l'état en introduisant des classes rares. Pour maintenir des effectifs équilibrés sans perdre trop d'information, on choisit alors d'utiliser les quintiles pour discrétiser la variable. Les modalités finales de Y sont donc : 13- (2551 individus), 14 (2150 individus), 15 (3244 individus), 16 (2331 individus) et 17+ (645 individus)

```
#Q03 : sexe
indiv_nr_Q03 <- escap[which(is.na(escap$Q03)),1]

#Q04
indiv_nr_Q04 <- escap[which(is.na(escap$Q04)),1]

#Q04 A et B: situation scolaire (A) et pro (B)
# Hypothèse : la situation scolaire et professionnelle s'excluent : NA = non concerné
# individus NR pour A et B
```

```

indiv_nr_Q04AB <- escap |>
  filter(is.na(Q04A) & is.na(Q04B)) |>
  select(A01)

#Q0A : individus répondants et non répondants exceptés individus NR pour A et B
indiv_nr_Q04A <- escap |>
  filter(!(A01 %in% indiv_nr_Q04AB$A01)) |>
  filter(is.na(Q04A)) |>
  dplyr::select(A01)

indiv_r_Q04A <- escap |>
  filter(!(is.na(Q04A))) |>
  dplyr::select(A01)

#Q0B : individus répondants et non répondants exceptés individus NR pour A et B
indiv_nr_Q04B <- escap |>
  filter(!(A01 %in% indiv_nr_Q04AB$A01)) |>
  filter(is.na(Q04B)) |>
  dplyr::select(A01)

indiv_r_Q04B <- escap |>
  filter(!(is.na(Q04B))) |>
  dplyr::select(A01)

#individus hors intersection entre répondre à Q04B et ne pas répondre à Q04A
diff_r_Q0B_nr_Q0A <- setdiff(
  indiv_nr_Q04A,
  indiv_r_Q04B
) #0

#individus hors intersection entre répondre à Q04A et ne pas répondre à Q04B
diff_r_Q0A_nr_Q0B <- setdiff(
  indiv_nr_Q04B,
  indiv_r_Q04A
) #0

#on en conclut que ormis les 76 : tous non concernés
pds_indiv_nr_Q04AB <- escap |>
  filter(A01 %in% indiv_nr_Q04AB$A01) |>
  select(pm17B)

pds_indiv_nr_Q04AB <- pds_indiv_nr_Q04AB$pm17B

#On impute une nouvelle modalité : 0 = "Non concerné" pour Q04A et Q04B

#Q04A
escap <- escap |>
  mutate(
    Q04A = ifelse(
      A01 %in% indiv_nr_Q04A$A01,
      0,
      Q04A
    )
  )

#Q04B

```

```

escap <- escap |>
  mutate(
    Q04B = ifelse(
      A01 %in% indiv_nr_Q04B$A01,
      0,
      Q04B
    )
  )

indiv_nr_Q04AB <- indiv_nr_Q04AB$A01

#Q04
# est-ce que le volume de NR peut être réduit avec info de Q04A et Q04B ?

indiv_r_Q04AB <- escap |>
  filter(!(is.na(Q04A)) & !(is.na(Q04B))) |>
  select(A01)

indiv_r_Q04AB <- indiv_r_Q04AB$A01

length(setdiff(indiv_nr_Q04, indiv_r_Q04AB))

```

```
[1] 19
```

```

# 19 indiv NR de Q04 ne sont pas parmi les indiv qui ont répondu à Q04A ou Q04B

# on impute la réponse en fonction des réponses cochées dans Q04A et Q04B
escap <- escap |>
  mutate(
    Q04 = ifelse(is.na(Q04A), NA, 1)
  ) |>
  mutate(
    Q04 = ifelse(
      !(Q04B %in% c(0,NA)),
      2,
      Q04
    )
  )

#ici on a une hypothèse : travailler (ou rechercher) = arrêter ses études
#néanmoins on a vu que les non réponses dans Q04A et Q04B étaient vraisemblablement des
#réponses de non concerné : donc pour nos individus cette hypothèse est crédible.

#Q05 : redoublement
any(is.na(escap$Q05))#TRUE

```

```
[1] TRUE
```

```

indiv_nr_Q05 <- escap |>
  filter(is.na(Q05)) |>
  select(A01)

indiv_nr_Q05 <- indiv_nr_Q05$A01

setdiff(
  indiv_nr_Q05,
  indiv_nr_Q04AB
)

```

```
) #36 sur 40 sont pas parmi les non répondants de A et B
```

```
[1] 44056 25021 23876 40027 22499 30068 36015 22442 21725 20859 5457 14473
[13] 22679 6840 36656 13624 24448 33544 27627 35446 33384 9751 28791 23735
[25] 31439 14702 5593 17216 4751 34189 2561 15369 26649 1575 22524 35792
```

Ici, on s'est intéressé aux variables Q04, Q04A et Q04B. C'est à dire les variables traitant de la situation scolaire et/ou professionnelle. Avec quelques manipulations, on a pu réduire la non-réponse conditionnelle présente dans ces variables. C'est à dire que dans la grande majorité des cas, un répondant qui travaille ne fait pas d'étude et inversement.

```
#Q06A et B
```

```
any(is.na(escap$Q06A))#TRUE
```

```
[1] TRUE
```

```
indiv_nr_Q06A <- escap |>
  filter(is.na(Q06A)) |>
  select(A01)
```

```
indiv_nr_Q06A <- indiv_nr_Q06A$A01
```

```
any(is.na(escap$Q06B))#TRUE
```

```
[1] TRUE
```

```
indiv_nr_Q06B <- escap |>
  filter(is.na(Q06B)) |>
  select(A01)
```

```
indiv_nr_Q06B <- indiv_nr_Q06B$A01
```

```
#intersections entre non répondants ?
```

```
setdiff(indiv_nr_Q06B, indiv_nr_Q06A)
```

```
[1] 41866 43295 5446 1227 24573 37430 44372 27802 39272 8687 6546 21045
[13] 568 6840 5619 4027 11761 32849 32858 11989 17696 36669 34033 18718
[25] 31439 21474 13329 32940 14563 26711 41099 43047 4598 12877
```

```
setdiff(indiv_nr_Q06A, indiv_nr_Q06B)
```

```
[1] 41453 2404 33254 35373 3985 44429 12427 17375 31562 23165 12046 8536
[13] 15563 28009 7865 21072 9067 19770 23054 29854 29141 636 36871 42539
[25] 34968 10797 20892 38571
```

```
#globalement, peu d'individus n'ont pas répondu aux deux questions
```

```
#Q08
```

```
any(is.na(escap$Q08))#TRUE
```

```
[1] TRUE
```

```
indiv_nr_Q08 <- escap |>
  filter(is.na(Q08)) |>
  select(A01)
```

```
indiv_nr_Q08 <- indiv_nr_Q08$A01
```

```
#Q08C
```

```

indiv_nr_Q08C <- escap |>
  filter(is.na(Q08C)) |>
  select(A01)

indiv_nr_Q08C <- indiv_nr_Q08C$A01

#individus qui ont pas connus leur parent pour X raison => réduire NR des réponses sur parents

indiv_sans_parents <- escap |>
  filter(Q08C %in% c(3,4)) |>
  select(A01)

indiv_sans_parents <- indiv_sans_parents$A01 #538

indiv_sans_pere <- escap |>
  filter(Q09A1 == 7) |>
  select(A01)

indiv_sans_pere <- indiv_sans_pere$A01 #554 individus sans père

indiv_sans_mere <- escap |>
  filter(Q09B1 == 7) |>
  select(A01)

indiv_sans_mere <- indiv_sans_mere$A01 #157 individus sans mere

#Q09A1
indiv_nr_Q09A1 <- escap |>
  filter(is.na(Q09A1)) |>
  select(A01)

indiv_nr_Q09A1 <- indiv_nr_Q09A1$A01 #402 non répondants

#indiv qui ne savent pas :
indiv_nsp_pere <- escap |>
  filter(Q09A1 == 6) |>
  select(A01)

indiv_nsp_pere <- indiv_nsp_pere$A01

#Q09B1
indiv_nr_Q09B1 <- escap |>
  filter(is.na(Q09B1)) |>
  select(A01)

indiv_nr_Q09B1 <- indiv_nr_Q09B1$A01

#indiv qui ne savent pas :
indiv_nsp_mere <- escap |>
  filter(Q09B1 == 6) |>
  select(A01)

indiv_nsp_mere <- indiv_nsp_mere$A01

```

```

#Q10A1
escap <- escap |>
  mutate(
    Q10A1 = ifelse(
      A01 %in% indiv_sans_pere,
      9,
      Q10A1
    )
  )

escap <- escap |>
  mutate(
    Q10A1 = ifelse(
      A01 %in% indiv_sans_parents,
      9,
      Q10A1
    )
  )

#nouvelle modalité : ne sait pas
escap <- escap |>
  mutate(
    Q10A1 = ifelse(
      A01 %in% indiv_nsp_pere,
      0,
      Q10A1
    )
  )

indiv_nr_Q10A1 <- escap |>
  filter(is.na(Q10A1)) |>
  select(A01)

indiv_nr_Q10A1 <- indiv_nr_Q10A1$A01

escap$Q10A1 <- as.factor(escap$Q10A1)


#Q10B1
escap <- escap |>
  mutate(
    Q10B1 = ifelse(
      A01 %in% indiv_sans_mere,
      9,
      Q10B1
    )
  )

escap <- escap |>
  mutate(
    Q10B1 = ifelse(
      A01 %in% indiv_sans_parents,
      9,
      Q10B1
    )
  )

```

```

    )
  )

#nouvelle modalité : ne sait pas
escap <- escap |>
  mutate(
    Q10B1 = ifelse(
      A01 %in% indiv_nsp_mere,
      0,
      Q10B1
    )
  )

indiv_nr_Q10B1 <- escap |>
  filter(is.na(Q10B1)) |>
  select(A01)

indiv_nr_Q10B1 <- indiv_nr_Q10B1$A01

escap$Q10B1 <- as.factor(escap$Q10B1)

#B08A
any(is.na(escap$B08A))#TRUE

```

```
[1] TRUE
```

```

indiv_nr_B08A <- escap |>
  filter(is.na(B08A)) |>
  select(A01)

indiv_nr_B08A <- indiv_nr_B08A$A01 #1400 non répondants

#ajout d'une nouvelle modalité : 0 = non concerné
escap <- escap |>
  mutate(
    B08A = ifelse(
      A01 %in% indiv_sans_pere,
      0,
      B08A
    )
  )

indiv_nr_B08A <- escap |>
  filter(is.na(B08A)) |>
  select(A01)

indiv_nr_B08A <- indiv_nr_B08A$A01

escap <- escap |>
  mutate(
    B08A = ifelse(
      A01 %in% indiv_sans_parents,
      0,
      B08A
    )
  )

```

```

indiv_nr_B08A <- escap |>
  filter(is.na(B08A)) |>
  select(A01)

indiv_nr_B08A <- indiv_nr_B08A$A01

escap$B08A <- as.factor(escap$B08A)

#B08B
any(is.na(escap$B08B))#TRUE

```

[1] TRUE

```

indiv_nr_B08B <- escap |>
  filter(is.na(B08B)) |>
  select(A01)

indiv_nr_B08B <- indiv_nr_B08B$A01 #1050 non répondants

#on ajoute une modalité: 0 = non concerné
escap <- escap |>
  mutate(
    B08B = ifelse(
      A01 %in% indiv_sans_mere,
      0,
      B08B
    )
  )

indiv_nr_B08B <- escap |>
  filter(is.na(B08B)) |>
  select(A01)

indiv_nr_B08B <- indiv_nr_B08B$A01

escap <- escap |>
  mutate(
    B08B = ifelse(
      A01 %in% indiv_sans_parents,
      0,
      B08B
    )
  )

indiv_nr_B08B <- escap |>
  filter(is.na(B08B)) |>
  select(A01)

indiv_nr_B08B <- indiv_nr_B08B$A01

escap$B08B <- as.factor(escap$B08B)

```

En observant les différentes modalités de certaines questions, on a à nouveau pu réduire les non-réponses dans des variables liées. Notamment, lorsqu'un répondant déclare sa mère comme décédée ou perdue de vue, il ne peut logiquement pas déclarer sa consommation d'alcool.

2 Statistiques descriptives

tests d'indépendance du Khi2

```
# 1. Liste des variables explicatives à tester face à Y
vars <- c(
  "Q03", "Q04", "Q04A", "Q04B", "Q05",
  "Q06A", "Q06B", "Q08", "Q08C", "Q09A1",
  "Q09B1", "Q10A1", "Q10B1", "B08A", "B08B"
)

# 2. Exécution des tests du Chi2 et extraction des statistiques
chi2_results <- map_dfr(vars, function(v) {
  # Récupération du test du Chi2 (ignore silencieusement les avertissements d'effectifs faibles)
  test <- suppressWarnings(chisq.test(escap$Y, escap[[v]]))

  tibble(
    Variable      = v,
    `Statistique (Chi2)` = round(unname(test$statistic), 2),
    `Degrés de liberté` = unname(test$parameter),
    `p-value`      = test$p.value,
    Significatif   = ifelse(test$p.value < 0.05, "Oui", "Non")
  )
})

# 3. Formater la p-value pour un affichage propre (ex: "< 0.001")
chi2_results <- chi2_results %>%
  mutate(
    `p-value` = ifelse(`p-value` < 0.001, "< 0.001", sprintf("%.4f", `p-value`))
  )

# Affichage du tableau dans la sortie de la cellule
chi2_results
```

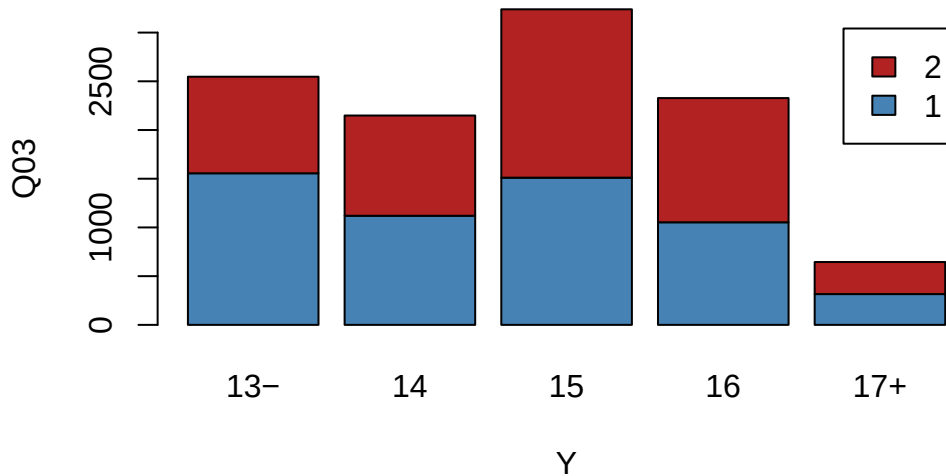
```
# A tibble: 15 x 5
  Variable `Statistique (Chi2)` `Degrés de liberté` `p-value` Significatif
  <chr>          <dbl>             <int> <chr>      <chr>
1 Q03           161.              4 < 0.001   Oui
2 Q04            21.5             4 < 0.001   Oui
3 Q04A           35.7            12 < 0.001   Oui
4 Q04B           30.6            16 0.0154   Oui
5 Q05           147.              4 < 0.001   Oui
6 Q06A           42.2             8 < 0.001   Oui
7 Q06B           55.9             8 < 0.001   Oui
8 Q08            22.5            16 0.1271   Non
9 Q08C           14.8            16 0.5357   Non
10 Q09A1          34.3            24 0.0796   Non
11 Q09B1          79.4            24 < 0.001   Oui
12 Q10A1         163.            36 < 0.001   Oui
13 Q10B1         128.            36 < 0.001   Oui
14 B08A          253.            20 < 0.001   Oui
15 B08B          238.            20 < 0.001   Oui
```

A priori, Y ne semble pas indépendante de chacun des prédicteurs pris à un un.

```
#Barplot
#Q03 : sexe
tab <- table(escap[[2]], escap[[18]])
```

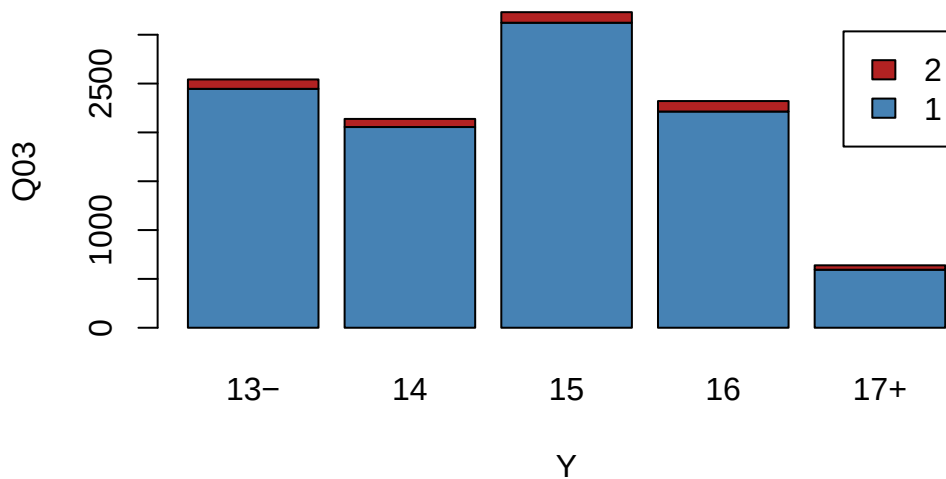
```
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick"), main = "Répartition du sexe selon l'âge de
```

Répartition du sexe selon l'âge de première consommation



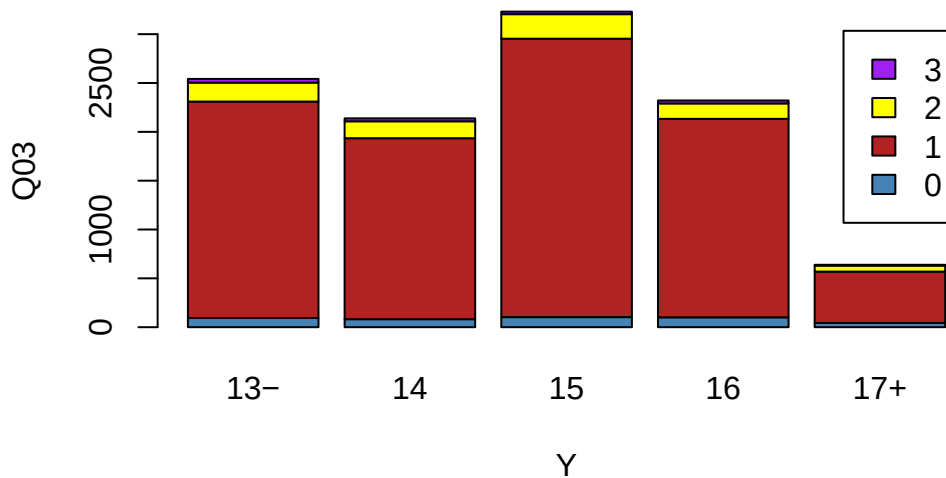
```
#Q04 : Situation
tab <- table(escap[[3]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick"), main = "Répartition de la situation personnelle
```

Répartition de la situation personnelle selon l'âge de première consommation



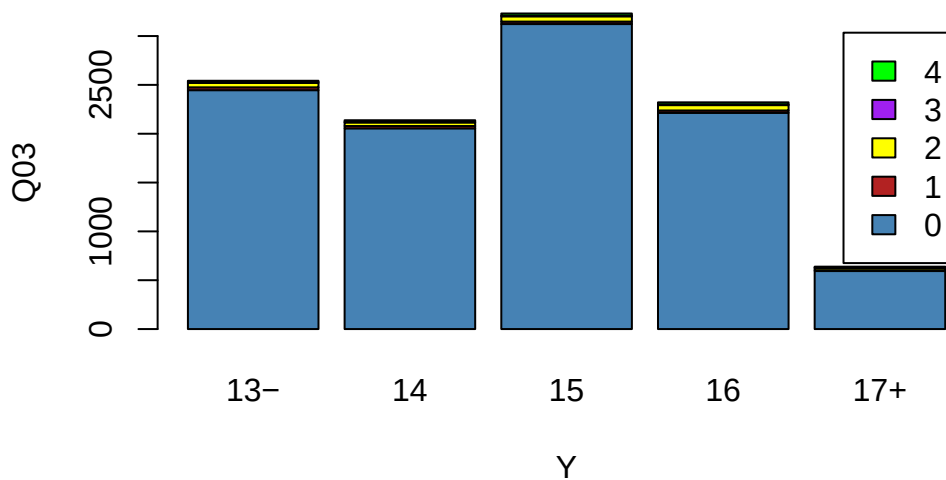
```
#Q04A : Situation scolaire
tab <- table(escap[[4]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple"), main = "Répartition de la situation scolaire
```

de la situation vis-à-vis des études selon l'âge de première



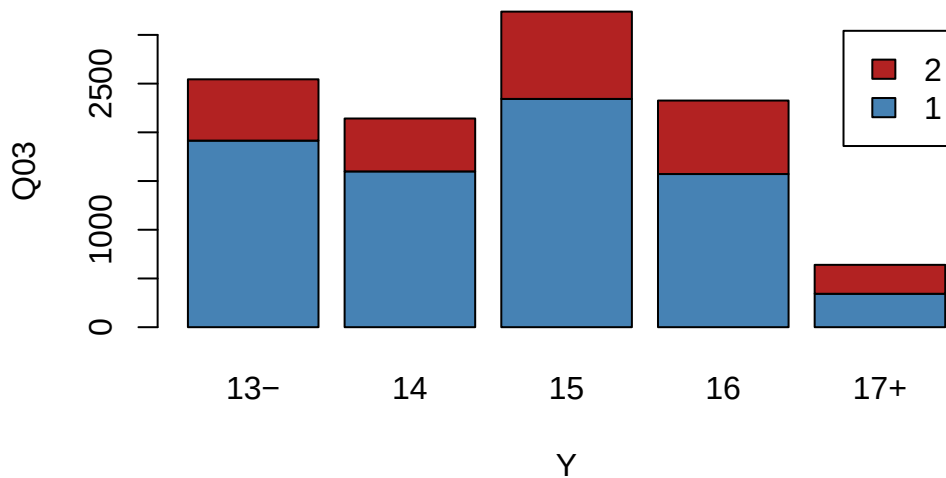
```
#Q04B : Situation pro
tab <- table(escap[[5]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "green"), main = "Répart
```

partition de la situation pro selon l'âge de première consom



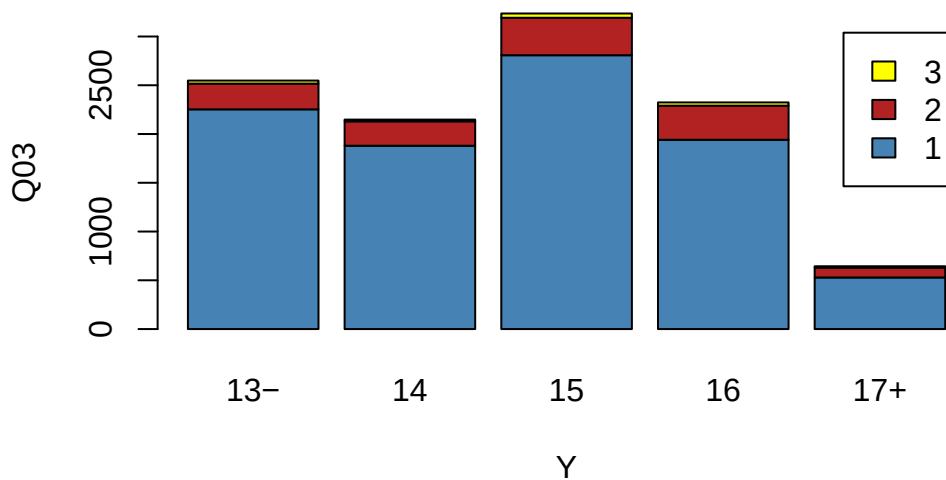
```
#Q05 : redoublement
tab <- table(escap[[6]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick"), main = "Répartition du redoublement selon l
```

Répartition du redoublement selon l'âge de première consommation



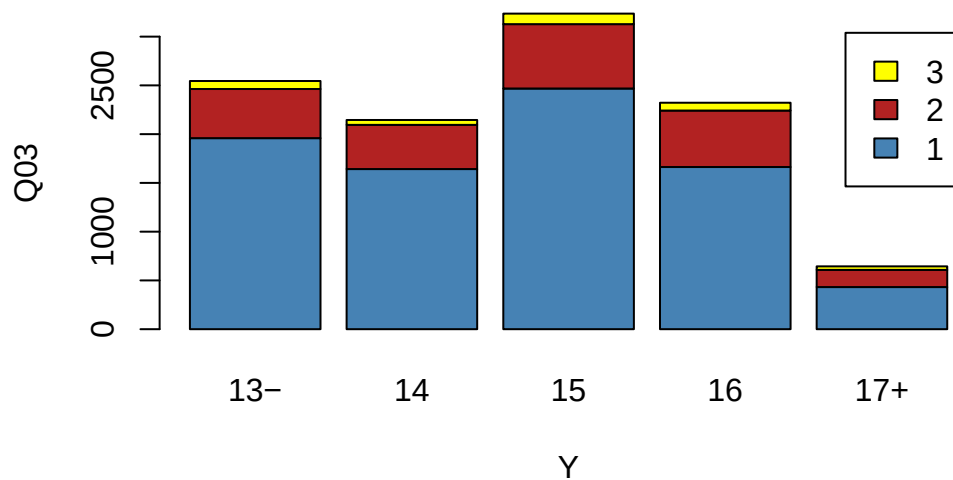
```
#Q06A
tab <- table(escap[[7]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow"), main = "Répartition du niveau de difficulté à lire selon l'âge de première consommation")
```

Répartition du niveau de difficulté à lire selon l'âge de première consommation



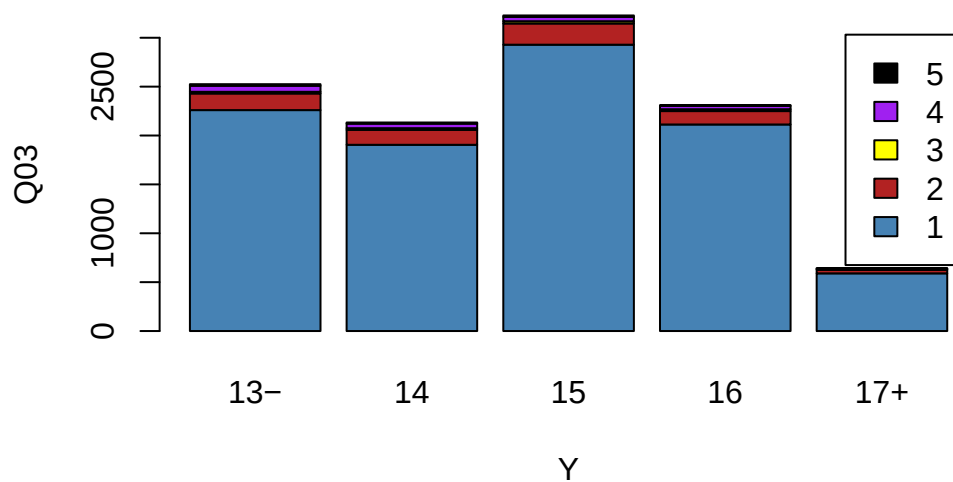
```
#Q06B
tab <- table(escap[[8]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow"), main = "Répartition du niveau de difficulté à lire selon l'âge de première consommation")
```

on du niveau de difficulté à écrire selon l'âge de première cc



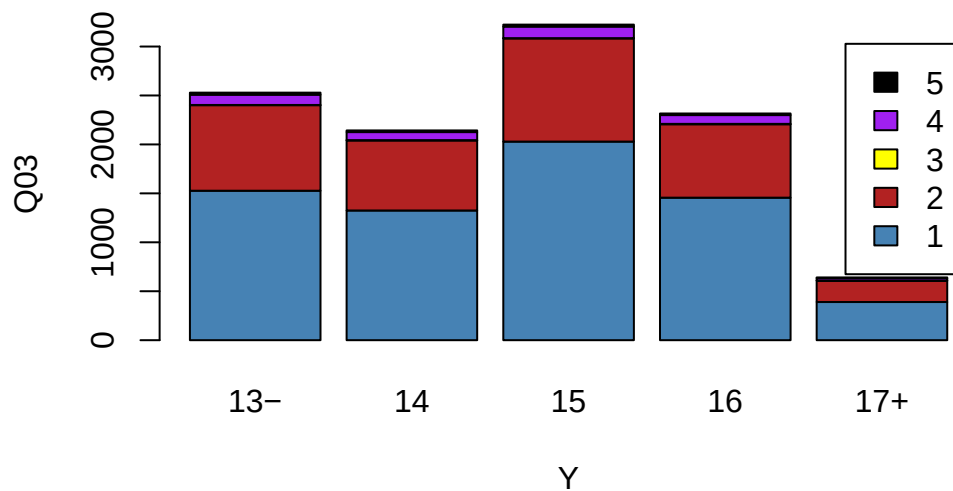
```
#Q08
tab <- table(escap[[9]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "black"), main = "Répart
```

Répartition du lieu de vie selon l'âge de première consommation



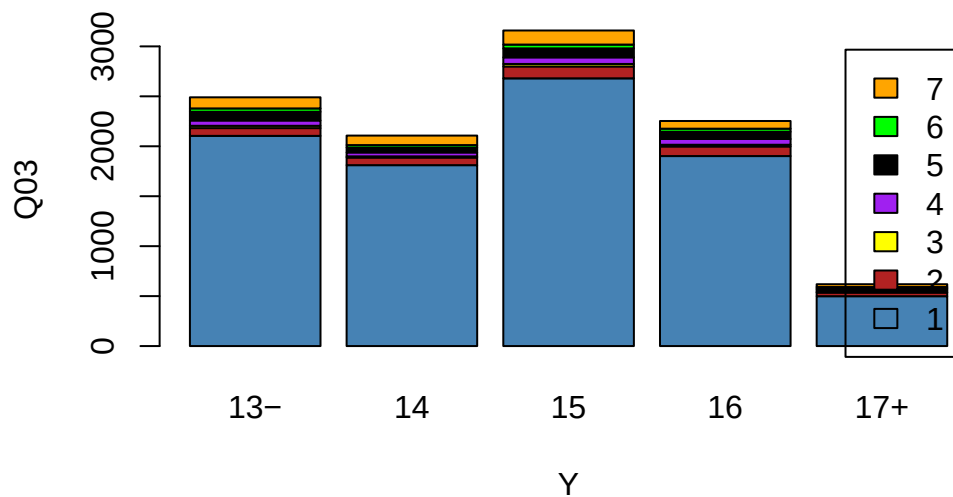
```
#Q08C
tab <- table(escap[[10]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "black"), main = "Répart
```

ion de la configuration parentale selon l'âge de première co



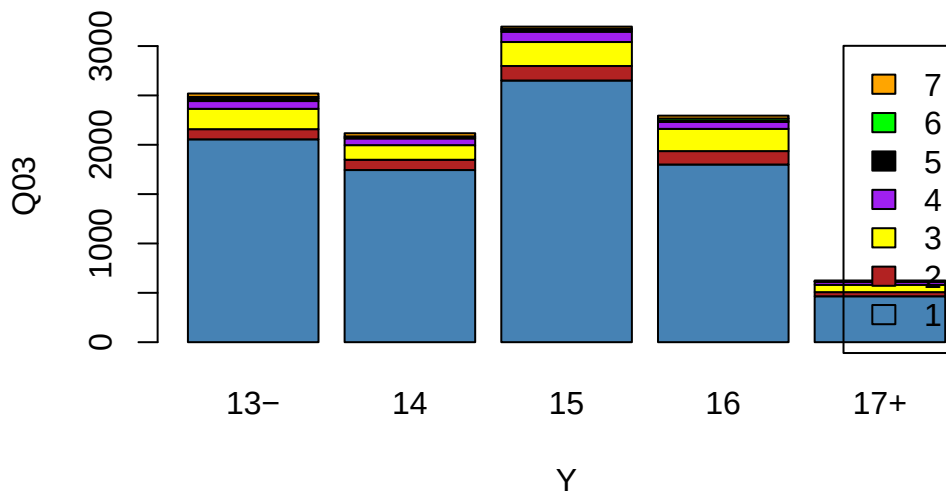
```
#Q09A1
tab <- table(escap[[11]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "black", "green", "orange"))
```

rtition de la situation du père selon l'âge de première conso



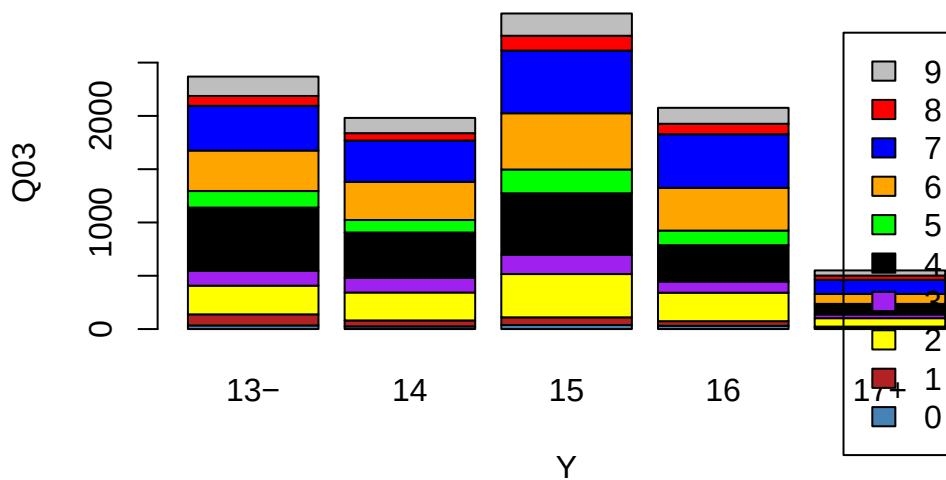
```
#Q09B1
tab <- table(escap[[12]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "black", "green", "orange"))
```

partition de la situation de la mère selon l'âge de première cons



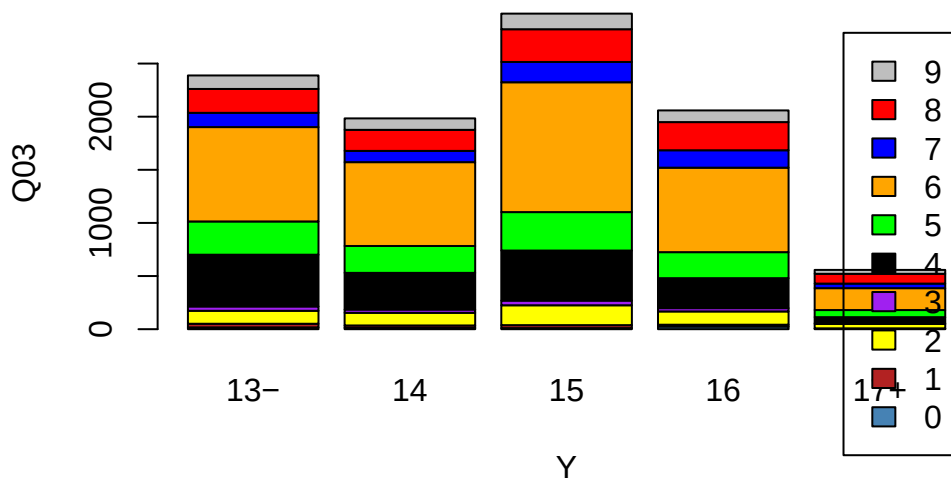
```
#Q10A1
tab <- table(escap[[13]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "black", "green", "orange"))
```

partition de la PCS du père selon l'âge de première consom



```
#Q10B1
tab <- table(escap[[14]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "black", "green", "orange"))
```

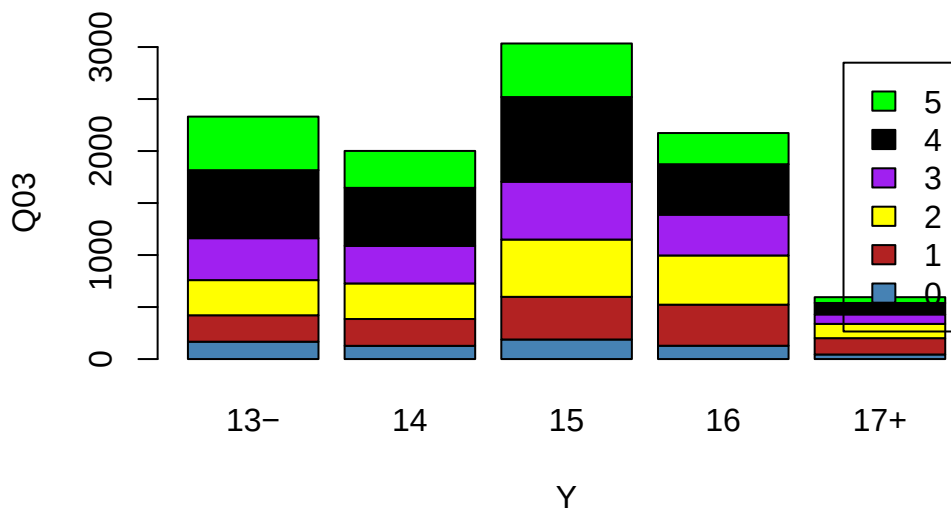
artition de la PCS de la mère selon l'âge de première consor



#B08A

```
tab <- table(escap[[15]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "black", "green"), main :
```

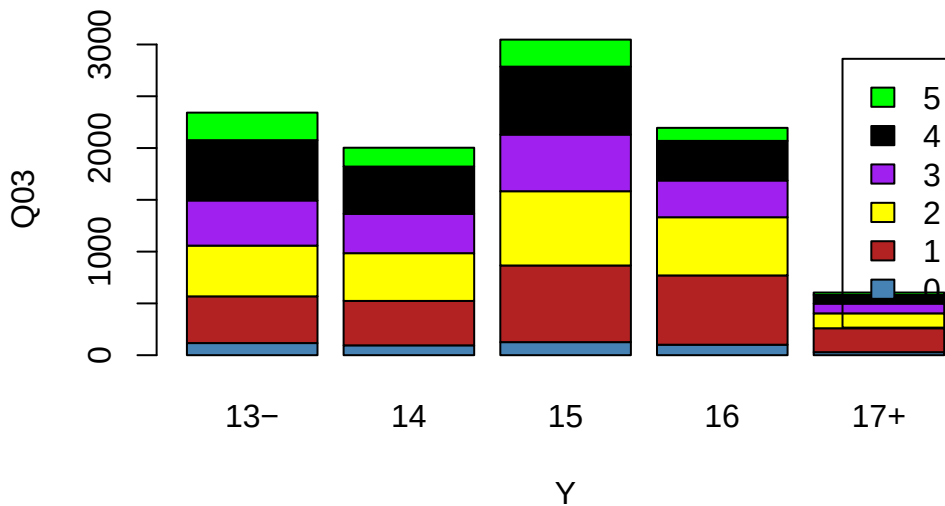
ion de la consommation du père selon l'âge de première coi



#B08B

```
tab <- table(escap[[16]], escap[[18]])
barplot(tab, legend = TRUE, col = c("steelblue", "firebrick", "yellow", "purple", "black", "green"), main :
```

on de la consommation de la mère selon l'âge de première c



L'inspection des graphiques en barre croisés ne soulève pas de lien évident entre chaque prédicteur et Y. Dans chacune des modalités de Y, les modalités des prédicteurs semblent se transposer dans des proportions relativement similaires. On ne peut donc pas tirer de premières conclusions.

3 Création d'un jeu de donnée imputé avec le package missmda

Pour pouvoir mener nos analyser et traiter la non réponse partielle dans nos individus, nous choisissons d'imputer les données manquantes, tant dans les prédicteurs que dans Y. Nous utilisons le package missmda.

```
df_MCA <- escap |> select(-c(pm17B,A01))
df_MCA <- df_MCA |> mutate(across(everything(), as.factor))
rownames(df_MCA) <- escap$A01

res <- MCA(escap)
# nb_comp <- estim_ncpMCA(df_MCA, ncp.max = 5, method.cv = "Kfold", nbsim = 20)
# nb_comp = 5
data_complete <- imputeMCA(df_MCA, 5)
data_complete <- data_complete$completeObs

write.csv(data_complete,
          file = "Impute_missMDA_gab.csv",
          row.names = FALSE)
```

4 Stacking de XGboost et RF

4.1 Préparation des données

```
data_rf <- read.csv2(
  "Impute_missMDA_gab.csv",
  sep = ",",
  stringsAsFactors = FALSE
)
data_rf <- data_rf |> mutate(across(1:16, as.factor))
```

```
poids_texte <- escap$pm17B

# 2. On remplace la virgule par un point avec gsub()
poids_propre <- gsub(",", ".", poids_texte)

# 3. On convertit en numeric et on l'assigne à data_rf
data_rf$pm17B <- as.numeric(poids_propre)
```

4.2 Séparation apprentissage / test

On utilise un seul découpage apprentissage/test pour les trois modèles afin que leurs performances soient comparables.

```
set.seed(42)

train_index <- createDataPartition(
  data_rf$Y,
  p = 0.70,
  list = FALSE
)

train_data <- data_rf[
  train_index,
  ,
  drop = FALSE
]

test_data <- data_rf[
  -train_index,
  ,
  drop = FALSE
]

classes <- levels(train_data$Y)
K <- length(classes)

# 5 folds communs à RF et XGBoost
set.seed(42)

folds <- createFolds(
  train_data$Y,
  k = 5,
  list = TRUE,
  returnTrain = FALSE
)

# Sécurisation préalable des types
train_data$Y <- as.factor(as.character(train_data$Y))
rf_features <- setdiff(names(train_data), c("Y", "pm17B"))
```

5 Paramètres de RF

```
# Variables explicatives
rf_features <- setdiff(
  names(train_data),
```

```

  c("Y", "pm17B")
)

# Hyperparamètres testés
rf_grid <- expand.grid(
  mtry = 1:7,
  num.trees = seq(50, 500, by = 50),
  min.node.size = c(1, 5, 10)
)

#####
# TUNING RANDOM FOREST PAR VALIDATION CROISEE
#####
rf_results <- rf_grid %>%
  mutate(
    cv_error = pmap_dbl(
      list(mtry, num.trees, min.node.size),
      function(m, ntree, node) {

        fold_errors <- map_dbl(
          folds,
          function(valid_index) {

            train_fold <- train_data[-valid_index, , drop = FALSE]
            valid_fold <- train_data[valid_index, , drop = FALSE]

            # Entraînement avec ranger
            model <- ranger(
              formula      = Y ~ . - pm17B,
              data          = train_fold,
              mtry          = m,
              num.trees     = ntree,
              min.node.size = node,
              case.weights  = train_fold$pm17B,
              probability   = FALSE
            )

            # Prédiction sur le pli de validation
            pred <- predict(
              model,
              data = valid_fold[, rf_features, drop = FALSE]
            )$predictions

            # Calcul du taux d'erreur pondéré par les poids de sondage
            weighted.mean(pred != valid_fold$Y, w = valid_fold$pm17B)
          }
        )
      }
    )
  )

# Moyenne des erreurs sur l'ensemble des plis
mean(fold_errors)
)
)

saveRDS(rf_results, "rf_results.rds")

```

```

# Meilleur modèle
best_rf <- rf_results %>%
  slice_min(
    cv_error,
    n = 1,
    with_ties = FALSE
  )

rf_mtry <- best_rf$mtry #3
rf_ntree <- best_rf$num.trees #500
rf_node <- best_rf$min.node.size #10

#erreur CV = 0.5821
cat(
  "Meilleur modèle : mtry =",
  rf_mtry,
  ", ntree =",
  rf_ntree,
  ", min.node.size =",
  rf_node,
  ", erreur_CV =",
  round(best_rf$cv_error, 4),
  "\n"
)

```

Meilleur modèle : mtry = 1 , ntree = 400 , min.node.size = 5 , erreur_CV = 0.7001

Meilleur modèle : mtry = 3 , ntree = 500 , min.node.size = 10 , erreur CV = 0.5821

Voici donc les paramètres optimaux pour notre Random Forest.

5.1 Paramètres de XGBoost

```

#####
# TUNING XGBOOST PAR VALIDATION CROISEE
#####

# Transformation des variables qualitatives en variables numériques
x_train <- model.matrix(
  Y ~ . - pm17B - 1,
  data = train_data
)

x_test <- model.matrix(
  Y ~ . - pm17B - 1,
  data = test_data
)

# Même ordre de colonnes pour train et test
x_test <- x_test[
  ,
  colnames(x_train),
  drop = FALSE
]

# Classes numériques 0, ..., K-1
y_train <- as.numeric(

```

```

train_data$Y
) - 1

y_test <- as.numeric(
  factor(
    test_data$Y,
    levels = classes
  )
) - 1

# Poids
w_train <- train_data$pm17B
w_test <- test_data$pm17B

#####
# HYPERPARAMETRES TESTES
#####

xgb_grid <- expand.grid(
  max_depth = c(2, 3, 4, 5, 6),
  nrounds = c(50, 100, 150, 200, 300),
  eta = c(0.01, 0.05, 0.10, 0.20)
)

#####
# VALIDATION CROISEE
#####

xgb_results <- xgb_grid %>%
  mutate(
    cv_error = pmap_dbl(
      list(
        max_depth,
        nrounds,
        eta
      ),
      function(depth, rounds, learning_rate) {

        fold_errors <- map_dbl(
          folds,
          function(valid_index) {

            train_index_fold <- setdiff(
              seq_len(nrow(train_data)),
              valid_index
            )

            dtrain <- xgb.DMatrix(
              x_train[
                train_index_fold,
              ],
              drop = FALSE
            ),
            label = y_train[
              train_index_fold
            ],

```

```

        weight = w_train[
            train_index_fold
        ]
    )

    dvalid <- xgb.DMatrix(
        x_train[
            valid_index,
            ,
            drop = FALSE
        ],
        label = y_train[
            valid_index
        ]
    )

    model <- xgb.train(
        params = list(
            objective = "multi:softprob",
            num_class = K,
            max_depth = depth,
            eta = learning_rate,
            eval_metric = "merror"
        ),
        data = dtrain,
        nrounds = rounds,
        verbose = 0
    )

    # Probabilités prédites
    pred_prob <- predict(
        model,
        dvalid
    )

    pred_prob <- matrix(
        pred_prob,
        ncol = K,
        byrow = TRUE
    )

    # Classe prédite
    pred_class <- max.col(
        pred_prob
    ) - 1

    # Erreur de classification
    mean(
        pred_class != y_train[
            valid_index
        ]
    )
}
)

mean(fold_errors)

```

```

    }
  )
)

saveRDS(xgb_results, "xgb_results.rds")

#####
# MEILLEURS PARAMETRES
#####

best_xgb <- xgb_results %>%
  slice_min(
    cv_error,
    n = 1,
    with_ties = FALSE
  )

xgb_max_depth <- best_xgb$max_depth
xgb_nrounds <- best_xgb$nrounds
xgb_eta <- best_xgb$eta

cat(
  "Meilleur modèle XGBoost : max_depth =",
  xgb_max_depth,
  ", nrounds =",
  xgb_nrounds,
  ", eta =",
  xgb_eta,
  ", erreur CV =",
  round(best_xgb$cv_error, 4),
  "\n"
)

```

Meilleur modèle XGBoost : max_depth = 5 , nrounds = 200 , eta = 0.1 , erreur CV = 0.7933

#meilleur param : max_depth = 5 , nrounds = 200 , eta = 0.1 , erreur CV = 0.7936

Meilleur modèle XGBoost : max_depth = 4 , nrounds = 300 , eta = 0.05 , erreur CV = 0.8301

Affichage des 10 meilleures combinaisons

```

xgb_results %>%
  arrange(cv_error) %>%
  head(10)

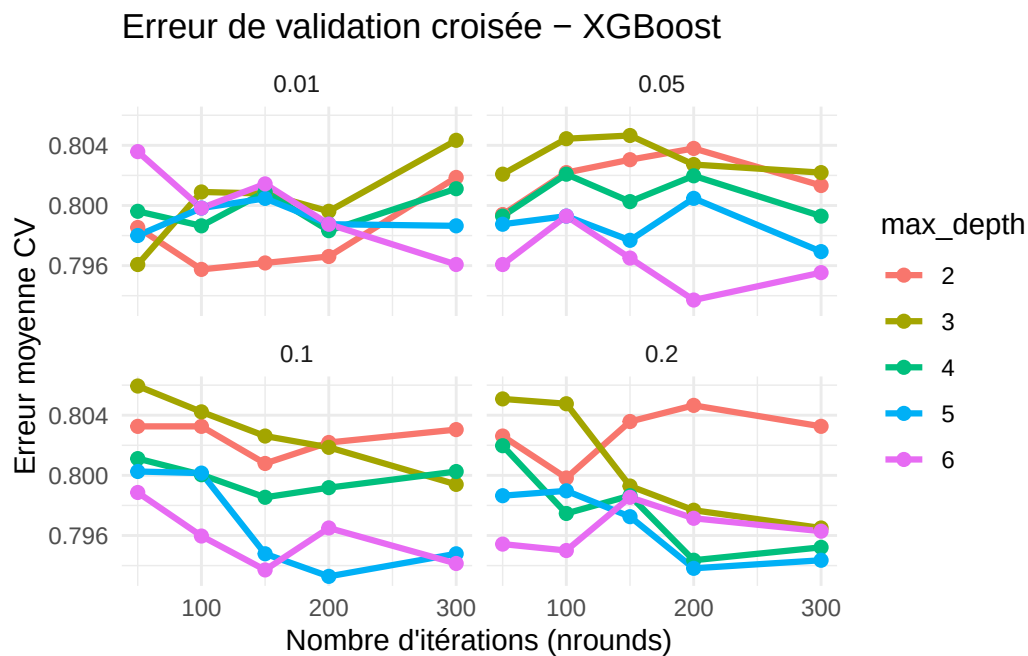
```

	max_depth	nrounds	eta	cv_error
1	5	200	0.10	0.7932841
2	6	150	0.10	0.7937122
3	6	200	0.05	0.7937126
4	5	200	0.20	0.7938188
5	6	300	0.10	0.7941408
6	5	300	0.20	0.7943550
7	4	200	0.20	0.7943565
8	5	300	0.10	0.7947853
9	5	150	0.10	0.7947858
10	6	100	0.20	0.7950021

Visualisation de l'erreur CV

```
ggplot(
```

```
xgb_results,
aes(
  x = nrounds,
  y = cv_error,
  color = factor(max_depth),
  group = max_depth
)
) +
geom_line(linewidth = 1.1) +
geom_point(size = 2) +
facet_wrap(~ eta) +
labs(
  title = "Erreur de validation croisée - XGBoost",
  x = "Nombre d'itérations (nrounds)",
  y = "Erreur moyenne CV",
  color = "max_depth"
) +
theme_minimal()
```



```
xgb_max_depth #5
```

```
[1] 5
```

```
xgb_nrounds #200
```

```
[1] 200
```

```
xgb_eta #0.1
```

```
[1] 0.1
```

Avec les paramètres de RF et XGBoost, on peut passer au stacking.

6 Stacking

6.1 Préparation des folds

On sépare TRAIN en plusieurs sous-ensembles de taille et de composition homogène (Folds) pour éviter les risques de data leakage entre les modèles de premier niveau (RF, XGBOOST) et le méta-modèle.

```
n_folds <- length(folds)

# Noms internes du stacking
rf_names <- paste0(
  "RF_C",
  seq_len(K)
)

xgb_names <- paste0(
  "XGB_C",
  seq_len(K)
)

oof_rf <- matrix(
  NA_real_,
  nrow = nrow(train_data),
  ncol = K,
  dimnames = list(
    NULL,
    rf_names
  )
)

oof_xgb <- matrix(
  NA_real_,
  nrow = nrow(train_data),
  ncol = K,
  dimnames = list(
    NULL,
    xgb_names
  )
)

# Predictions out of fold pour rf et xg

for (i in seq_along(folds)) {

  valid <- folds[[i]]

  train <- setdiff(
    seq_len(nrow(train_data)),
    valid
  )

  train_fold <- train_data[
    train,
    ,
    drop = FALSE
  ]

  valid_fold <- train_data[
```

```

    valid,
    ,
    drop = FALSE
  ]

# ----- RF -----

rf <- ranger(
  Y ~ . - pm17B,
  data = train_fold,
  num.trees = rf_ntree,
  mtry = rf_mtry,
  min.node.size = rf_node,
  probability = TRUE,
  case.weights = train_fold$pm17B,
  seed = 123 + i
)

oof_rf[
  valid,
] <- predict(
  rf,
  valid_fold[
    ,
    rf_features,
    drop = FALSE
  ]
)$predictions[
  ,
  classes,
  drop = FALSE
]

# ----- XGBoost -----

dtrain <- xgb.DMatrix(
  x_train[
    train,
    ,
    drop = FALSE
  ],
  label = y_train[
    train
  ],
  weight = w_train[
    train
  ]
)

dvalid <- xgb.DMatrix(
  x_train[
    valid,
    ,
    drop = FALSE
  ],

```

```

    label = y_train[
      valid
    ]
  )

  xgb <- xgb.train(
    params = list(
      objective = "multi:softprob",
      num_class = K,
      max_depth = xgb_max_depth,
      eta = xgb_eta,
      eval_metric = "mlogloss"
    ),
    data = dtrain,
    nrounds = xgb_nrounds,
    verbose = 0
  )

  oof_xgb[
    valid,
  ] <- matrix(
    predict(
      xgb,
      dvalid
    ),
    ncol = K,
    byrow = TRUE
  )
}

```

6.2 Méta-modèle multinomial

```

meta_data <- data.frame(
  oof_rf,
  oof_xgb,
  Y = train_data$Y,
  check.names = FALSE
)

meta_model <- multinom(
  Y ~ .,
  data = meta_data,
  trace = FALSE
)

```

6.3 RF et XGBoost finaux

Nous testons XGBoost et RF sur TEST avec leurs hyperparamètres optimisés par validation croisée sur TRAIN.

```

#####
# RF et XGBOOST finaux

# ----- RF -----

final_rf <- ranger(
  Y ~ . - pm17B,

```

```

data = train_data,
num.trees = rf_ntree,
mtry = rf_mtry,
min.node.size = rf_node,
probability = TRUE,
case.weights = w_train,
importance = "permutation",
seed = 999
)

pred_rf_test <- predict(
  final_rf,
  test_data[
    ,
    rf_features,
    drop = FALSE
  ]
)$predictions[
  ,
  classes,
  drop = FALSE
]

# ----- XGBoost -----

dtrain <- xgb.DMatrix(
  x_train,
  label = y_train,
  weight = w_train
)

dtest <- xgb.DMatrix(
  x_test,
  label = y_test,
  weight = w_test
)

final_xgb <- xgb.train(
  params = list(
    objective = "multi:softprob",
    num_class = K,
    max_depth = xgb_max_depth,
    eta = xgb_eta,
    eval_metric = "mlogloss"
  ),
  data = dtrain,
  nrounds = xgb_nrounds,
  verbose = 0
)

pred_xgb_test <- matrix(
  predict(
    final_xgb,
    dtest
  ),

```

```

ncol = K,
byrow = TRUE
)

colnames(pred_xgb_test) <- classes

```

6.4 Prédiction du stacking

```

colnames(pred_rf_test) <- rf_names
colnames(pred_xgb_test) <- xgb_names

meta_test <- data.frame(
  pred_rf_test,
  pred_xgb_test,
  check.names = FALSE
)

# Vérification importante
stopifnot(
  identical(
    names(meta_data)[
      names(meta_data) != "Y"
    ],
    names(meta_test)
  )
)

stacked_probs <- predict(
  meta_model,
  meta_test,
  type = "probs"
)

if (is.null(dim(stacked_probs))) {

  stacked_probs <- matrix(
    stacked_probs,
    ncol = K
  )
}

colnames(stacked_probs) <- classes

```

7 Premiers résultats

7.1 Classes prédites

```

rf_pred <- factor(
  classes[
    max.col(pred_rf_test)
  ],
  levels = classes
)

```

```
xgb_pred <- factor(
  classes[
    max.col(pred_xgb_test)
  ],
  levels = classes
)

stack_pred <- factor(
  classes[
    max.col(stacked_probs)
  ],
  levels = classes
)
```

7.2 Matrices de confusion

```
cm_rf <- confusionMatrix(
  rf_pred,
  test_data$Y,
  mode = "everything"
)

cm_xgb <- confusionMatrix(
  xgb_pred,
  test_data$Y,
  mode = "everything"
)

cm_stack <- confusionMatrix(
  stack_pred,
  test_data$Y,
  mode = "everything"
)
```

7.3 Comparaison des accuracies

```
accuracy_results <- data.frame(
  Modele = c(
    "Random Forest",
    "XGBoost",
    "Stacking"
  ),
  Accuracy = c(
    cm_rf$overall["Accuracy"],
    cm_xgb$overall["Accuracy"],
    cm_stack$overall["Accuracy"]
  )
)

print(accuracy_results)
```

	Modele	Accuracy
1	Random Forest	0.3990481
2	XGBoost	0.1966433
3	Stacking	0.4183367

Nos premiers résultats ne sont pas encourageants étant donné qu'on a des accuracy à moins de 0.5. Cela dit, avant une accuracy au delà de 0.4 pour notre stacking, la combinaison de modèle est bien bénéfique puisqu'on gagne en prédiction par rapport à random forest ou xgboost seuls.

7.4 Matrices de confusion pondérées utilisées plus tard

```
weighted_confusion <- function(  
  pred,  
  truth,  
  weights  
) {  
  
  xtabs(  
    weights ~ pred + truth,  
    data = data.frame(  
      pred = factor(  
        pred,  
        levels = classes  
      ),  
      truth = factor(  
        truth,  
        levels = classes  
      ),  
      weights = weights  
    )  
  )  
}  
  
weighted_cm_rf <- weighted_confusion(  
  rf_pred,  
  test_data$Y,  
  w_test  
)  
  
weighted_cm_xgb <- weighted_confusion(  
  xgb_pred,  
  test_data$Y,  
  w_test  
)  
  
weighted_cm_stack <- weighted_confusion(  
  stack_pred,  
  test_data$Y,  
  w_test  
)
```

8 Analyse Post-hoc des modèles

Étant donné que nos deux modèles et le stacking sont des modèles “boîtes noires”, on introduit des explicateurs dits “à posteriori”.

```
# Variables effectivement utilisées par le RF  
X_test_posthoc <- test_data[, rf_features, drop = FALSE]  
  
# Fonction de prédiction du RF :  
# on retourne les probabilités de chaque classe.
```

```

predict_rf_prob <- function(model, newdata) {
  predict(model, newdata)$predictions[, classes, drop = FALSE]
}

# Fonction permettant de sélectionner une classe pour les analyses
# PDP / ALE / Shapley locales.
target_class <- classes[which.max(table(train_data$Y))]
target_class

```

```
[1] "15"
```

Pour certaines des techniques d'analyse post-hoc qui nécessitent de viser une modalité, on se concentrera sur la modalité la plus peuplée : 15 ans.

8.1 3. OOB error du Random Forest

L'erreur OOB est une estimation de l'erreur de généralisation du Random Forest.

```

# Erreur OOB du Random Forest final
oob_error_rf <- final_rf$prediction.error

oob_error_rf

```

```
[1] 0.5433542
```

L'erreur OOB est assez haute. La performance de notre Random Forest est décevante.

9 4. Partial Dependence Plot (PDP)

Le PDP permet d'étudier l'effet moyen d'une variable sur la probabilité d'appartenir à une classe donnée. Ici, pour limiter les graphiques, on s'intéresse à 15 ans, la modalité la plus peuplée.

```

predictor_rf <- iml::Predictor$new(
  model = final_rf,
  data = X_test_posthoc,
  y = test_data$Y,
  predict.function = function(model, newdata) {
    predict_rf_prob(model, newdata)[, target_class]
  }
)

```

On sélectionne les variables les plus importantes du Random Forest pour les étudier.

```

top_pdp_vars <- names(
  head(
    sort(final_rf$variable.importance, decreasing = TRUE),
    4
  )
)

top_pdp_vars

```

```
[1] "B08A" "B08B" "Q10A1" "Q10B1"
```

On obtient les variables liées à la consommation d'alcool à la maison des parents et les CSP des parents.

```

# PDP pour les variables les plus importantes

pdp_results <- lapply(
  top_pdp_vars,

```

```

function(v) {
  effect <- iml::FeatureEffect$new(
    predictor_rf,
    feature = v,
    method = "pdp",
    grid.size = 20
  )

  effect
}
)

names(pdp_results) <- top_pdp_vars

# Créer les graphiques PDP
pdp_plots <- lapply(
  seq_along(pdp_results),
  function(i) {
    v <- top_pdp_vars[i]

    plot(pdp_results[[i]]) +
      ggtitle(
        paste(
          "Partial Dependence Plot -",
          v,
          "- classe",
          target_class
        )
      )
  }
)
)

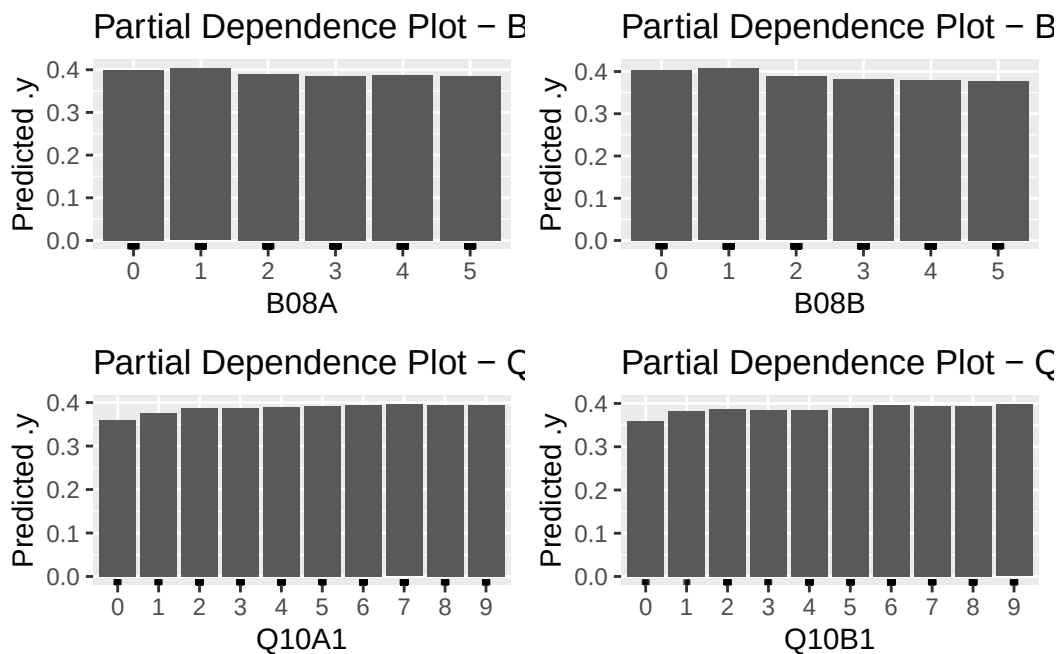
```

Warning: `aes\_string()` was deprecated in ggplot2 3.0.0.
 i Please use tidy evaluation idioms with `aes()`.
 i See also `vignette("ggplot2-in-packages")` for more information.
 i The deprecated feature was likely used in the iml package.
 Please report the issue at <<https://github.com/giuseppec/iml/issues>>.

```

gridExtra::grid.arrange(
  grobs = pdp_plots,
  ncol = 2,
  nrow = 2
)

```



Pour ces quatre variables, les PDP sont quasiment constant, peu importe la modalité. Donc la variation (de modalité) de ces variables a peu d'effet sur la prédiction.

10 5. Accumulated Local Effects (ALE)

```
ale_results <- lapply(
  top_pdp_vars,
  function(v) {
    effect <- iml::FeatureEffect$new(
      predictor_rf,
      feature = v,
      method = "ale",
      grid.size = 20
    )

    effect
  }
)

names(ale_results) <- top_pdp_vars

# Créer les 4 graphiques ALE
ale_plots <- lapply(
  seq_along(ale_results),
  function(i) {
    v <- top_pdp_vars[i]

    plot(ale_results[[i]]) +
      ggtitle(
        paste(
          "Accumulated Local Effects -",
          v,
          "- classe",

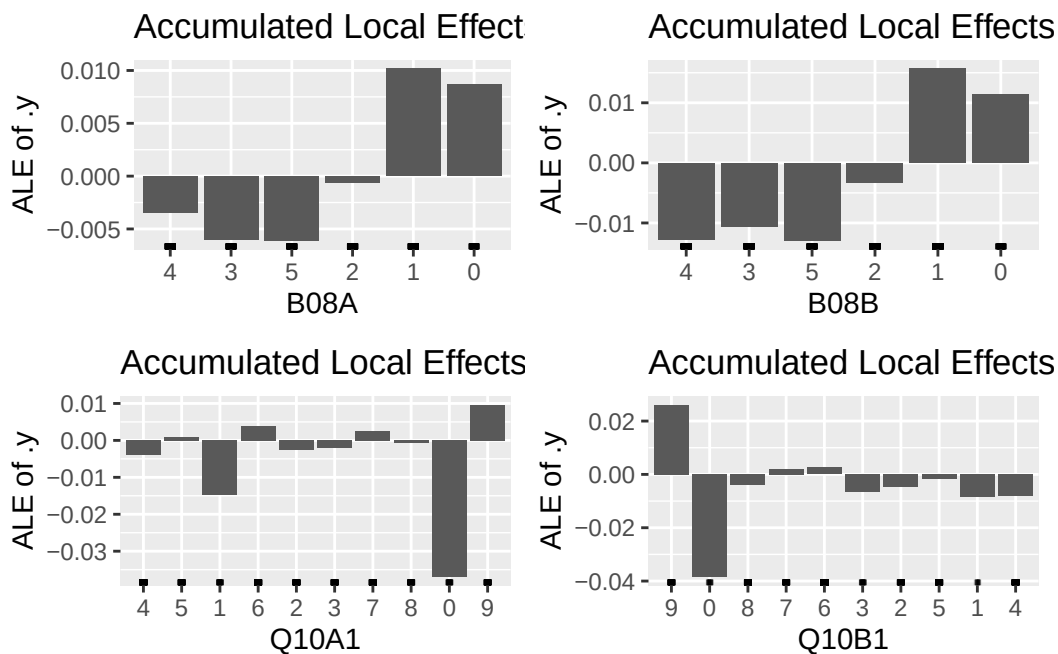
```

```

        target_class
      )
    )
  }
)

# Affichage en 2 x 2
gridExtra::grid.arrange(
  grobs = ale_plots,
  ncol = 2,
  nrow = 2
)

```



On voit que pour le père comme pour la mère, les mêmes modalités sont les plus impactantes pour la prédiction quand on prédit pour la modalité la plus peuplée : 15 ans.

Dans le cadre de la consommation d'alcool (B08A et B), la non consommation d'alcool (1) ainsi que être non concerné par la question (0) ont un effet positif sur la prédiction, contrairement aux consommation d'alcool plus importantes qui ont donc un effet négatif sur la prédiction.

Pour les CSP, ne être concerné par la question (modalité rajoutée pendant notre traitement de donnée) a un effet négatif important (0) sur la prédiction et la modalité : "Non concerné(e) (décédé(e), inconnu(e), perdu(e) de vue. . ." (9) a l'effet positif le plus important.

11 6. Permutation Feature Importance (PFI)

Le principe est de permuter une variable dans l'échantillon TEST puis de mesurer l'augmentation de l'erreur du modèle. On utilise le log-loss multiclass, ce qui tient compte de l'ensemble des probabilités prédites et pas seulement de la classe finalement choisie.

```

weighted_multiclass_logloss <- function(
  truth,
  probs,
  weights
) {

```

```

truth <- factor(truth, levels = classes)

# Probabilité attribuée à la vraie classe
p_true <- probs[
  cbind(
    seq_along(truth),
    match(truth, colnames(probs))
  )
]

# Eviter log(0)
p_true <- pmin(
  pmax(p_true, 1e-15),
  1 - 1e-15
)

sum(
  weights * (-log(p_true))
) / sum(weights)
}

```

Erreur de référence sur TEST :

```

# Prédiction de référence
probs_rf_test <- predict_rf_prob(
  final_rf,
  X_test_posthoc
)

baseline_logloss <- weighted_multiclass_logloss(
  truth = test_data$Y,
  probs = probs_rf_test,
  weights = w_test
)

baseline_logloss

```

```
[1] 1.413701
```

Calcul du PFI :

```

set.seed(123)

n_permutations <- 10

pfi_results <- map_dfr(
  rf_features,
  function(v) {

    permutation_losses <- numeric(n_permutations)

    for (b in seq_len(n_permutations)) {

      X_perm <- X_test_posthoc

      X_perm[[v]] <- sample(
        X_perm[[v]],
        size = nrow(X_perm),

```

```

      replace = FALSE
    )

    probs_perm <- predict_rf_prob(
      final_rf,
      X_perm
    )

    permutation_losses[b] <-
      weighted_multiclass_logloss(
        truth = test_data$Y,
        probs = probs_perm,
        weights = w_test
      )
  }

  tibble(
    Variable = v,
    PFI_mean = mean(permutation_losses) - baseline_logloss,
    PFI_sd = sd(permutation_losses),
    PFI_min = min(permutation_losses) - baseline_logloss,
    PFI_max = max(permutation_losses) - baseline_logloss
  )
} |>
  arrange(desc(PFI_mean))

```

pfi_results

```

# A tibble: 15 x 5
  Variable PFI_mean  PFI_sd  PFI_min PFI_max
  <chr>      <dbl>    <dbl>    <dbl>    <dbl>
1 B08A      0.00829  0.00115  0.00613  0.00957
2 B08B      0.00723  0.000945 0.00561  0.00902
3 Q03       0.00443  0.000546 0.00335  0.00536
4 Q05       0.00429  0.000750 0.00334  0.00567
5 Q10B1     0.00399  0.000558 0.00297  0.00463
6 Q10A1     0.00358  0.000440 0.00266  0.00431
7 Q09B1     0.00354  0.000584 0.00297  0.00447
8 Q06B      0.00302  0.000527 0.00223  0.00375
9 Q09A1     0.00278  0.000376 0.00201  0.00342
10 Q06A     0.00187  0.000426 0.00114  0.00272
11 Q08C     0.00144  0.000343 0.000553 0.00172
12 Q04A     0.00119  0.000273 0.000727 0.00160
13 Q08      0.00102  0.000367 0.000506 0.00172
14 Q04      0.000992 0.000366 0.000334 0.00162
15 Q04B     0.000776 0.000272 0.000364 0.00118

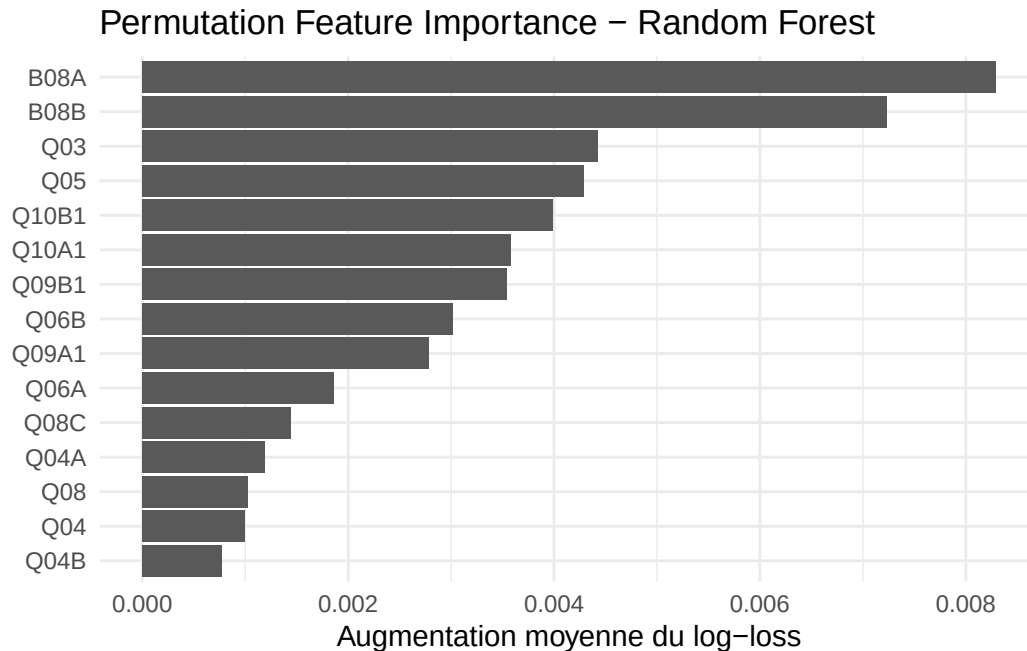
```

```

ggplot(
  head(pfi_results, 15),
  aes(
    x = reorder(Variable, PFI_mean),
    y = PFI_mean
  )
) +
  geom_col() +
  coord_flip() +

```

```
labs(
  title = "Permutation Feature Importance - Random Forest",
  x = NULL,
  y = "Augmentation moyenne du log-loss"
) +
theme_minimal()
```



Les variables dont la permutation réduit le plus notre prédiction sont les consommations d'alcool des parents : B08A et B08B.

12 7. LOFO - Leave One Feature Out

Le LOFO consiste à retirer une variable, ré-estimer le modèle puis comparer sa performance au modèle complet.

```
lofo_results <- map_dfr(
  rf_features,
  function(v) {

    features_without_v <- setdiff(
      rf_features,
      v
    )

    formula_lofo <- as.formula(
      paste(
        "Y ~",
        paste(features_without_v, collapse = " + ")
      )
    )

    rf_lofo <- ranger(
      formula_lofo,
      data = train_data,
      num.trees = rf_ntree,
```

```

    mtry = min(
      rf_mtry,
      length(features_without_v)
    ),
    min.node.size = rf_node,
    probability = TRUE,
    case.weights = w_train,
    seed = 999
  )

  probs_lofo <- predict(
    rf_lofo,
    test_data[, features_without_v, drop = FALSE]
  )$predictions[, classes, drop = FALSE]

  logloss_lofo <- weighted_multiclass_logloss(
    truth = test_data$Y,
    probs = probs_lofo,
    weights = w_test
  )

  tibble(
    Variable = v,
    LogLoss_complet = baseline_logloss,
    LogLoss_sans_variable = logloss_lofo,
    LOFO = logloss_lofo - baseline_logloss
  )
}
) |>
  arrange(desc(LOFO))

lofo_results

```

```

# A tibble: 15 x 4
  Variable LogLoss_complet LogLoss_sans_variable LOFO
  <chr>      <dbl>          <dbl>      <dbl>
1 B08B      1.41            1.42  0.00446
2 B08A      1.41            1.42  0.00406
3 Q10B1     1.41            1.42  0.00290
4 Q03       1.41            1.42  0.00259
5 Q09A1     1.41            1.42  0.00225
6 Q05       1.41            1.42  0.00216
7 Q10A1     1.41            1.42  0.00192
8 Q09B1     1.41            1.42  0.00154
9 Q06B      1.41            1.42  0.00139
10 Q08      1.41            1.41  0.00129
11 Q08C     1.41            1.41  0.000900
12 Q06A     1.41            1.41  0.000708
13 Q04A     1.41            1.41  0.000622
14 Q04B     1.41            1.41 -0.000870
15 Q04      1.41            1.41 -0.00219

```

```

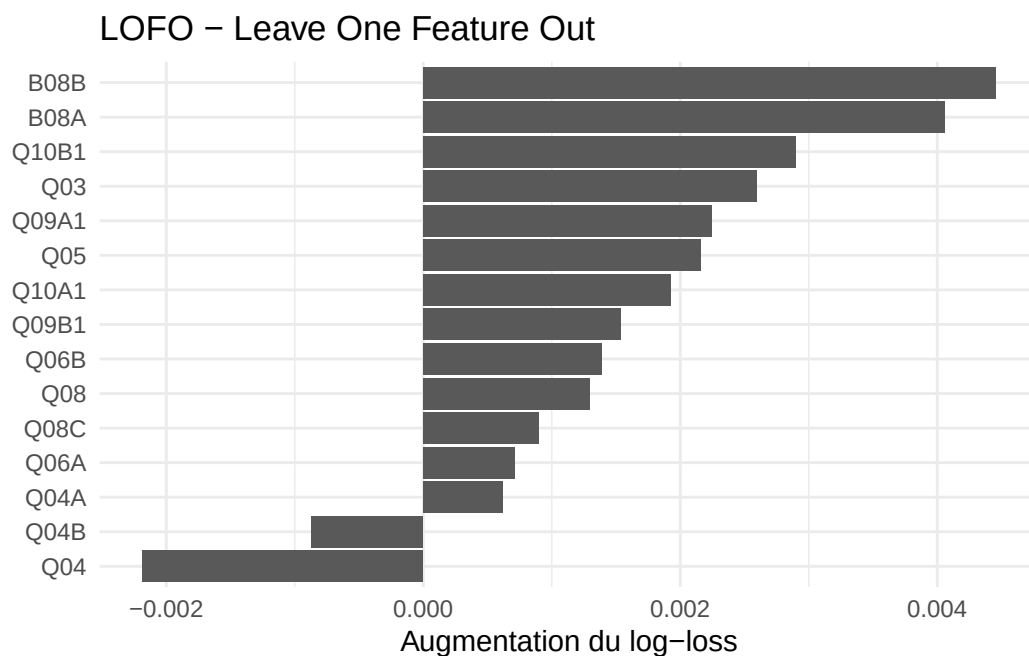
ggplot(
  head(lofo_results, 15),
  aes(
    x = reorder(Variable, LOFO),
    y = LOFO
  )
)

```

```

)
) +
  geom_col() +
  coord_flip() +
  labs(
    title = "LOFO - Leave One Feature Out",
    x = NULL,
    y = "Augmentation du log-loss"
  ) +
  theme_minimal()

```



A nouveau, perdre B08A et B baisserait le plus la performance de notre modèle. Nous ne sommes pas très surpris de l'importance répétée de ces variables puisque la consommation d'alcool est un phénomène connu de mimétisme social.

13 8. Valeurs de Shapley : explication locale

```

shapley_predictor <- iml::Predictor$new(
  model = final_rf,
  data = X_test_posthoc,
  y = test_data$Y,
  predict.function = function(model, newdata) {
    predict_rf_prob(model, newdata)[, target_class]
  }
)

# Première observation du TEST
x_interest <- X_test_posthoc[1, , drop = FALSE]

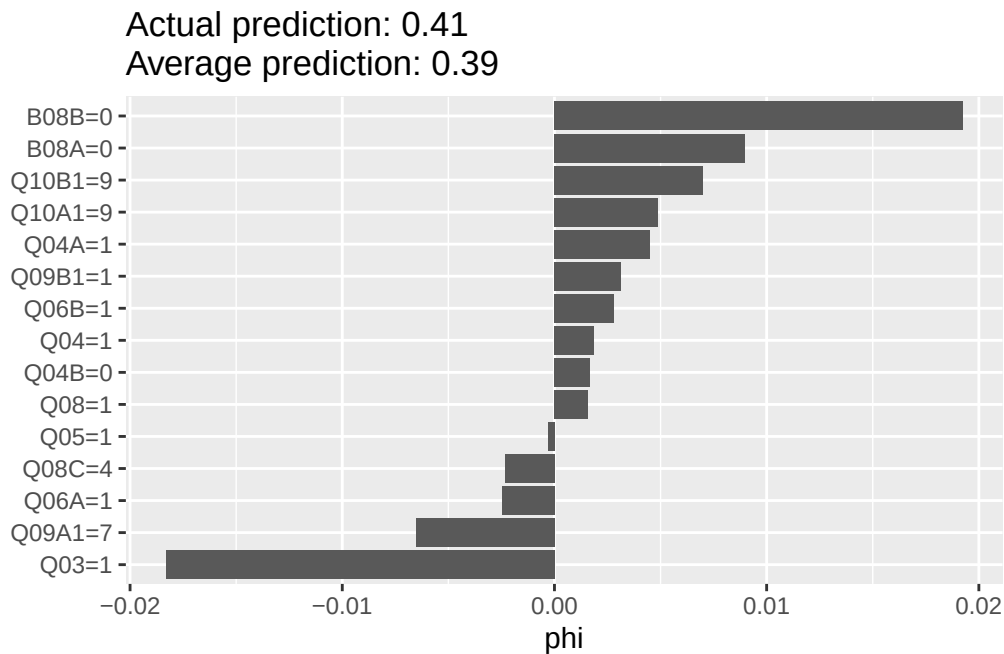
set.seed(123)

shapley_one <- iml::Shapley$new(
  predictor = shapley_predictor,

```

```
x.interest = x_interest,
sample.size = 100
)

plot(shapley_one)
```



En regardant les deux valeurs extrêmes :

- Être non concerné par la question sur la consommation d'alcool de la mère contribue plus qu'aucune autre modalité à orienter le modèle vers une prédiction du premier verre d'alcool à 15 ans.
- Le fait que l'individu répondant soit un homme tire plutôt le modèle vers d'autres classes que 15 ans. Probablement parce que les répondants hommes boivent quand ils sont plus jeunes.

13.1 Résultats finaux à comparer

```
print(cm_rf)
```

Confusion Matrix and Statistics

Prediction	Reference				
	13-	14	15	16	17+
13-	8	3	4	6	2
14	0	0	0	0	0
15	762	622	1528	757	182
16	13	20	19	57	9
17+	0	0	0	0	0

Overall Statistics

```
Accuracy : 0.399
95% CI : (0.3838, 0.4144)
No Information Rate : 0.3885
P-Value [Acc > NIR] : 0.08905
```

Kappa : 0.0276

McNemar's Test P-Value : NA

Statistics by Class:

	Class: 13-	Class: 14	Class: 15	Class: 16	Class: 17+
Sensitivity	0.010217	0.0000	0.98517	0.06951	0.00000
Specificity	0.995326	1.0000	0.04834	0.98077	1.00000
Pos Pred Value	0.347826	NaN	0.39678	0.48305	NaN
Neg Pred Value	0.804737	0.8384	0.83688	0.80305	0.95165
Precision	0.347826	NA	0.39678	0.48305	NA
Recall	0.010217	0.0000	0.98517	0.06951	0.00000
F1	0.019851	NA	0.56572	0.12154	NA
Prevalence	0.196142	0.1616	0.38853	0.20541	0.04835
Detection Rate	0.002004	0.0000	0.38277	0.01428	0.00000
Detection Prevalence	0.005762	0.0000	0.96468	0.02956	0.00000
Balanced Accuracy	0.502771	0.5000	0.51676	0.52514	0.50000

RF semble prédire la modalité 15 (ans) peut importe l'individu et a beaucoup de mal à prédire les autres classes. Malgré tout, cela nous donne une accuracy proche de 0.4, plus performante que le hasard.

```
print(cm_xgb)
```

Confusion Matrix and Statistics

	Reference				
Prediction	13-	14	15	16	17+
13-	160	126	303	150	34
14	146	139	325	171	49
15	149	98	288	169	41
16	176	133	296	154	25
17+	152	149	339	176	44

Overall Statistics

Accuracy : 0.1966
95% CI : (0.1844, 0.2093)
No Information Rate : 0.3885
P-Value [Acc > NIR] : 1

Kappa : 0.0022

McNemar's Test P-Value : <2e-16

Statistics by Class:

	Class: 13-	Class: 14	Class: 15	Class: 16	Class: 17+
Sensitivity	0.20434	0.21550	0.18569	0.18780	0.22798
Specificity	0.80897	0.79355	0.81278	0.80139	0.78521
Pos Pred Value	0.20699	0.16747	0.38658	0.19643	0.05116
Neg Pred Value	0.80646	0.83997	0.61103	0.79239	0.95243
Precision	0.20699	0.16747	0.38658	0.19643	0.05116
Recall	0.20434	0.21550	0.18569	0.18780	0.22798
F1	0.20566	0.18847	0.25087	0.19202	0.08357
Prevalence	0.19614	0.16157	0.38853	0.20541	0.04835
Detection Rate	0.04008	0.03482	0.07214	0.03858	0.01102
Detection Prevalence	0.19364	0.20792	0.18662	0.19639	0.21543

Balanced Accuracy 0.50666 0.50453 0.49923 0.49460 0.50659

Avec une accuracy à 0.2 et une matrice de confusion aussi bien répartie, notre xgboost semble être à peine meilleur que le hasard.

```
print(cm_stack)
```

Confusion Matrix and Statistics

		Reference				
Prediction		13-	14	15	16	17+
13-	137	137	83	121	56	14
14	0	0	0	0	0	0
15	606	519	1366	597	153	
16	40	43	64	167	26	
17+	0	0	0	0	0	0

Overall Statistics

Accuracy : 0.4183
95% CI : (0.403, 0.4338)
No Information Rate : 0.3885
P-Value [Acc > NIR] : 6.342e-05

Kappa : 0.1008

Mcnemar's Test P-Value : NA

Statistics by Class:

	Class: 13-	Class: 14	Class: 15	Class: 16	Class: 17+
Sensitivity	0.17497	0.0000	0.8807	0.20366	0.00000
Specificity	0.91462	1.0000	0.2319	0.94546	1.00000
Pos Pred Value	0.33333	NaN	0.4215	0.49118	NaN
Neg Pred Value	0.81960	0.8384	0.7537	0.82119	0.95165
Precision	0.33333	NA	0.4215	0.49118	NA
Recall	0.17497	0.0000	0.8807	0.20366	0.00000
F1	0.22948	NA	0.5701	0.28793	NA
Prevalence	0.19614	0.1616	0.3885	0.20541	0.04835
Detection Rate	0.03432	0.0000	0.3422	0.04183	0.00000
Detection Prevalence	0.10296	0.0000	0.8119	0.08517	0.00000
Balanced Accuracy	0.54479	0.5000	0.5563	0.57456	0.50000

Notre stacking a la meilleure accuracy. On voit à sa matrice de confusion qu'il est assez proche de RF mais a amélioré sa prévision sur les 13 ans et moins et les 16 ans.

En l'état, les performances prédictives de nos modèles sont assez faibles, surtout pour XGBOOST.

En étudiant les matrices de confusion, on observe que XGboost a tendance à prédire de manière relativement uniforme les individus dans les modalités de Y, au contraire de la forêt aléatoire qui prédit quasiment par le mode (15 ans) pour la majorité des individus. La combinaison des deux modèles par le stacking améliore très légèrement la qualité de prédiction mais évince totalement les modalités 14 et 17+ et continue de prédire en grande majorité la modalité 15.

Nous retenons de l'analyse post hoc que pour la forêt aléatoire, les variables associées à la consommation des parents et dans une moindre mesure, la catégorie sociale sont prépondérantes pour prédire l'âge de première consommation.

Concernant Xgboost, les variables enregistrant le plus de gain compte tenu de la fonction objectif sont La consommation d'alcool du père et le fait d'avoir redoublé. Contrairement à RF qui marque une nette séparation, le gain des prédicteur chez Xgboost décroît de manière plus continue. Enfin, le classement

Ces observations doivent être considérées avec prudence compte tenu de la faible propension des modèles à bien prédire.

14 Idées pour améliorer prédiction:

1)

Pour simplifier le problème d'apprentissage et gagner en prédictibilité, je propose de considérer le problème du point de vue d'une classification binaire. Pour déterminer les regroupements des modalités, nous proposons de considérer plusieurs découpages, d'entraîner un modèle sur chacun et comparer les performances de prédiction entre découpages uniquement sur TRAIN. Une fois le meilleur découpage binaire sélectionné, on conserve cette binarisation et on regarde les performances sur TEST.

2)

En ayant discrétisé Y sur les quintiles, on a peut être éclaté des individus aux profils et à l'âge de première consommation homogènes, qu'on a alors mélangé avec d'autres distincts, ce qui a brouillé les pistes. Je propose de rediscrétiser les modalités de Y sur la base d'un clustering. NB : le clustering devra se faire uniquement sur TRAIN (sinon on apprend indirectement des données TEST). Pour ce faire on peut recourir à une méthode basée sur l'acm et implémentable avec **FactoMineR**, via la fonction "chclust".

3)

il existe un package pour faire des random forest spécialisé pour prédire une variable ordinale: **ordinalForest**

A noter que nous avons tester un modèle en retirant les non-répondants à notre variable Y plutôt qu'en considérant qu'il s'agit de répondants non-concernés et que la performance ne s'est pas améliorée.